



给 Nix 用户的 λ -演算基础

λ -Calculus Basics for Nix Users

Yinfeng

2026-06-13

关于我

Yinfeng



github.com/linyinfeng

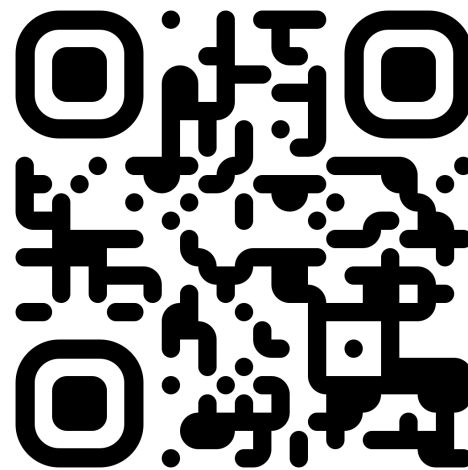
也算是个研究程序设计语言的。

λ-演算 - 开始之前

你可能想要：



获取本幻灯片



在线λ-演算解释器

lambdacalc.dev

λ-演算?

- 回想我第一次尝试使用 NixOS 的时候，对 Nix 语言非常不适应；
- 学了 λ-演算后，第二次接触 NixOS 语言时，我的感想完全变了。

这 Nix 语言不就是：

λ-演算 + bool + int + string + attrset + derivation¹

味太对了。

¹包含 string context。

目录

1. λ -演算简介
2. 用 λ -演算编程
3. 语法糖
4. λ -演算的扩展
5. λ -演算的形式语义
6. λ -演算的求值顺序
7. 不动点
8. 理解 Nix 语法和库
 - `let & rec & lib.fix`
 - Overlay & NixOS modules
9. 扩展知识
 8. 不动点组合子
 9. 希尔伯特可判定性问题与邱奇-图灵论题
10. λ -演算的实现

λ -演算简介



图 1: 阿隆佐·邱奇

- Lambda calculus
 - 一种理想化的编程语言
 - 一个形式系统
 - 一种计算模型
- 阿隆佐·邱奇 (Alonzo Church, 1903–1995) 在 1930 年代提出

λ-演算简介

作为一种理想化的程序设计语言，λ-演算只有三种语法：

$$\begin{array}{lcl} M, N & ::= & x \quad \text{variable} \\ & | & \lambda x.M \quad \text{abstraction} \\ & | & MN \quad \text{application} \end{array}$$

λ-演算简介

作为一种理想化的程序设计语言，λ-演算只有三种语法：

$$\begin{array}{ll} M, N ::= x & \text{variable} \\ & | \lambda x.M \text{ abstraction} \\ & | MN \text{ application} \end{array}$$

1	x	# variable	nix
2	x: M	# abstraction	
3	M N	# application	

λ-演算简介

作为一种理想化的程序设计语言，λ-演算只有三种语法：

$$\begin{array}{lcl} M, N ::= & x & \text{variable} \\ & | \lambda x.M & \text{abstraction} \\ & | MN & \text{application} \end{array}$$

1	x	# variable	nix
2	x: M	# abstraction	
3	M N	# application	

Abstraction 在这里就是函数的另一种说法。

λ-演算简介

作为一种理想化的程序设计语言，λ-演算只有三种语法：

$$\begin{array}{ll} M, N ::= x & \text{variable} \\ & | \lambda x.M \text{ abstraction} \\ & | MN \text{ application} \end{array}$$

1	x	# variable	nix
2	x: M	# abstraction	
3	M N	# application	

Abstraction 在这里就是函数的另一种说法。

M, N 被叫作 λ-项 (terms)，也被称作 lambda 表达式，也就是“程序”。

λ-演算简介 – 书写规则

$$M, N = x \mid \lambda x.M \mid MN$$

λ-项的定义是抽象语法树，有时我们需要在书写中添加一些括号来明确项到底是怎么结合的。

比如：

$$(xy)z \neq x(yz)$$

$$\lambda x.(xy) \neq (\lambda x.x)y$$

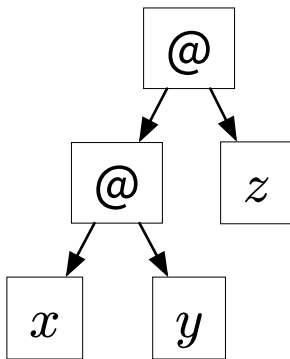
我们需要规定形如 xyz 或 $\lambda x.xy$ 的项究竟是哪一个项。

λ -演算简介 - 书写规则

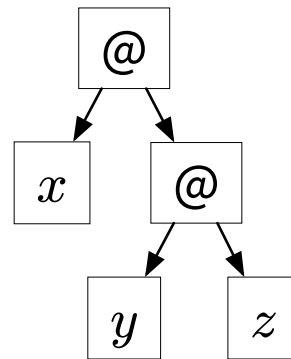
$$M, N = x \mid \lambda x.M \mid MN$$

规则 1: 函数应用 (applications) 是左结合的

$$xyz = (xy)z$$



$$x(yz)$$

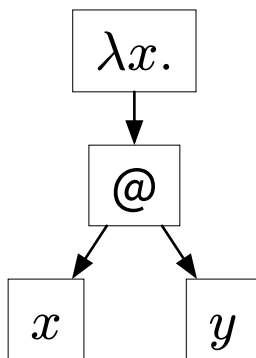


λ -演算简介 - 书写规则

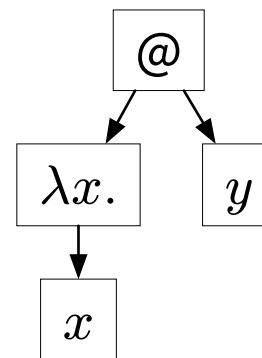
$$M, N = x \mid \lambda x.M \mid MN$$

规则 2: 抽象 (abstractions) 的优先级比应用低 ($\lambda x.$ 作用到最远的位置)

$$\lambda x.xy = \lambda x.(xy)$$



$$(\lambda x.x)y$$



λ-演算简介

在 λ-演算中，所有抽象（abstractions）都是“**匿名函数**”。在许多程序设计语言中都可以找到类似物。

$\lambda x.M$ 在不同语言中的类似物：

语言	写法
Nix	<code>x: M</code>
Haskell	<code>\x → M</code>
Scheme	<code>(lambda (x) M)</code>
Python	<code>lambda x: M</code>
Rust	<code> x M</code>
JavaScript	<code>function(x) { return M; }</code>

语言	写法
JavaScript	<code>x ⇒ M</code>
C++	<code>[] (auto x) { return M; }</code>
Kotlin	<code>fun(x) = M</code>
Swift	<code>{ x in M }</code>

λ-演算简介 – 例子

通过几个例子，我们可以在不形式化的情况下简单理解 λ-演算的语义。

1. 恒等函数 I (identity): $I \stackrel{\text{def}}{=} \lambda x.x$

$$\underline{I}y = \underline{(\lambda x.x)}y \rightarrow_{\beta} y$$

$$\underline{II} = \underline{(\lambda x.x)(\lambda x.x)} \rightarrow_{\beta} (\lambda x.x)$$

$$\underline{III} = \underline{(\lambda x.x)(\lambda y.y)(\lambda z.z)} \rightarrow_{\beta} \underline{(\lambda y.y)(\lambda z.z)} \rightarrow_{\beta} (\lambda z.z)$$

- 此处 I 并不是一个语法/变量，只是为了方便，将 $\lambda x.x$ 简写成了 I 。
- 我们并没有定义 Iy 中的 y 是什么，它是一个**自由变量**。
- 可以通过类比其他语言中的匿名函数进行理解。

λ-演算简介 – 例子

通过几个例子，我们可以在不形式化的情况下简单理解 λ-演算的语义。

2. 常数函数 K (const): $K \stackrel{\text{def}}{=} \lambda x. \lambda y. x$

$$\underline{K}ab = \underline{(\lambda x. \lambda y. x)a}b \rightarrow_{\beta} \underline{(\lambda y. a)}b \rightarrow_{\beta} a$$

λ-演算简介 – 例子

通过几个例子，我们可以在不形式化的情况下简单理解 λ-演算的语义。

3. ω 组合子: $\omega \stackrel{\text{def}}{=} \lambda x.xx$

$$\underline{\omega}I = \underline{(\lambda x.xx)(\lambda y.y)} \rightarrow_{\beta} \underline{(\lambda y.y)(\lambda z.z)} \rightarrow_{\beta} (\lambda z.z)$$

4. Ω 组合子: $\Omega \stackrel{\text{def}}{=} \omega\omega$

$$\underline{\Omega} = \underline{\omega}\omega = \underline{(\lambda x.xx)\omega} \rightarrow_{\beta} \omega\omega = \Omega$$

注意到，虽然我们将 I 定义为 $\lambda x.x$ ，但将 x 改名为 y 所得到的 $\lambda y.y$ 与 $\lambda x.x$ 等价，这被称为 α -等价 (α -equivalence)，将在之后涉及。

用 λ -演算编程

只有函数和函数应用的程序设计语言，真的能写程序吗？

能，而且它图灵完备。

- 布尔值
 - 自然数
 - 对 (pair)
 - 列表
 - ...
- 这些数据结构和它们的操作都可以用 λ -项表示。
- 我们将会介绍一种表示方式：
- 邱奇编码² (Church encoding)

²https://en.wikipedia.org/wiki/Church_encoding

用 λ -演算编程 – 邱奇编码 – 布尔值

$$T = \lambda x. \lambda y. x$$

$$F = \lambda x. \lambda y. y$$

$$\text{if_then_else} = \lambda b. \lambda t. \lambda f. b t f$$

第一次见可能难以理解，让我们看看以下这两个 λ -项等于什么。

- $\text{if_then_else } T M N$
- $\text{if_then_else } F M N$

用 λ -演算编程 – 邱奇编码 – 布尔值 – 验证

$$\text{if_then_else } T \ MN = \underline{(\lambda b. \lambda t. \lambda f. b t f)(\lambda x. \lambda y. x)} MN$$

$$\rightarrow_{\beta} \underline{(\lambda t. \lambda f. (\lambda x. \lambda y. x) t f)} MN$$

$$\rightarrow_{\beta} \underline{(\lambda f. (\lambda x. \lambda y. x) M f)} N$$

$$\rightarrow_{\beta} \underline{(\lambda x. \lambda y. x) M} N$$

$$\rightarrow_{\beta} \underline{(\lambda y. M)} N$$

$$\rightarrow_{\beta} M$$

$$\text{if_then_else } F \ MN \xrightarrow{\beta}^* N$$

用 λ -演算编程 – 邱奇编码 – 布尔值

很自然的，我们可以定义 $\text{not} \stackrel{\text{def}}{=} \lambda b. \text{if_then_else } bFT$.

$$\begin{aligned}\text{not} &= \lambda b. \text{if_then_else } bFT \\ &= \lambda b. (\lambda t. \lambda f. btf) bFT \\ &\rightarrow_{\beta} \lambda b. (\lambda t. \lambda f. btf) FT \\ &\rightarrow_{\beta} \lambda b. (\lambda f. bFf) T \\ &\rightarrow_{\beta} \lambda b. bFT\end{aligned}$$

可以注意到，不同于上一页，我们在函数未被应用前就进行了“求值”。我们将在讲到求值顺序的时候讨论这件事情。

用 λ -演算编程 – 邱奇编码 – 布尔值 – 构造与使用

一个数据结构有哪些要素？

$$T = \lambda x. \lambda y. x$$

$$F = \lambda x. \lambda y. y$$

$$\text{if_then_else} = \lambda b. \lambda t. \lambda f. b t f$$

用 λ -演算编程 – 邱奇编码 – 布尔值 – 构造与使用

一个数据结构有哪些要素？

- 如何构造 – T 和 F
- 如何使用 – `if_then_else`

$$T = \lambda x. \lambda y. x$$

$$F = \lambda x. \lambda y. y$$

$$\text{if_then_else} = \lambda b. \lambda t. \lambda f. b t f$$

让我们再次仔细观察它们的定义。

用 λ -演算编程 – 邱奇编码 – 布尔值 – 构造与使用

一个数据结构有哪些要素？

- 如何构造 – T 和 F
- 如何使用 – `if_then_else`

$$T = \lambda x. \lambda y. x$$

$$F = \lambda x. \lambda y. y$$

$$\text{if_then_else} = \lambda b. \lambda t. \lambda f. b t f$$

让我们再次仔细观察它们的定义。

- T 接受两个参数，返回第一个； F 也接受两个参数，返回第二个。
- `if_then_else` 实际上在效果上和恒等函数 I 是类似的。

$\lambda x. M x$ 可以被视作等价于 M 。

这其实被称为 η -等价，将在之后涉及。

$$\text{if_then_else} \rightarrow_{\eta} \lambda b. \lambda t. \lambda f. b t f$$

$$\rightarrow_{\eta} \lambda b. \lambda t. b t$$

$$\rightarrow_{\eta} \lambda b. b = I$$

用 λ -演算编程 – 邱奇编码 – 布尔值 – 构造与使用

邱奇编码的独特之处在于，构造出的数据结构用函数直接编码了“使用”。

$$\text{and} \stackrel{\text{def}}{=} \lambda a. \lambda b. abF$$

$$\text{or} \stackrel{\text{def}}{=} \lambda a. \lambda b. aTb$$

因为 a 本身就等价于 Ia 等价于 $\text{if_then_else } a$ 。不难验证：

$$\text{and } TT \rightarrow_{\beta}^* T$$

$$\text{or } TT \rightarrow_{\beta}^* T$$

$$\text{and } TF \rightarrow_{\beta}^* F$$

$$\text{or } TF \rightarrow_{\beta}^* T$$

$$\text{and } FT \rightarrow_{\beta}^* F$$

$$\text{or } FT \rightarrow_{\beta}^* T$$

$$\text{and } FF \rightarrow_{\beta}^* F$$

$$\text{or } FF \rightarrow_{\beta}^* F$$

用 λ -演算编程 – 邱奇编码 – 自然数

一个非常重要的邱奇编码是自然数（包含零）的编码。

用 $0, 1, 2, \dots, n, \dots$ 表示数学中的自然数。在数上加横线，用 $\bar{0}, \bar{1}, \bar{2}, \dots, \bar{n}, \dots$ 表示它在 λ -演算中的编码。

$$\bar{0} \stackrel{\text{def}}{=} \lambda f. \lambda x. x$$

$$\bar{1} \stackrel{\text{def}}{=} \lambda f. \lambda x. f x$$

$$\bar{2} \stackrel{\text{def}}{=} \lambda f. \lambda x. f(f x)$$

...

用 λ -演算编程 – 邱奇编码 – 自然数

一个非常重要的邱奇编码是自然数（包含零）的编码。

用 $0, 1, 2, \dots, n, \dots$ 表示数学中的自然数。在数上加横线，用 $\bar{0}, \bar{1}, \bar{2}, \dots, \bar{n}, \dots$ 表示它在 λ -演算中的编码。

$$\bar{0} \stackrel{\text{def}}{=} \lambda f. \lambda x. x$$

$$\bar{1} \stackrel{\text{def}}{=} \lambda f. \lambda x. f x$$

$$\bar{2} \stackrel{\text{def}}{=} \lambda f. \lambda x. f(f x)$$

...

$$\bar{n} \stackrel{\text{def}}{=} \lambda f. \lambda x. f^n x$$

我们用 $f^n x$ 表示 $\underbrace{f(f(\dots(f(fx))\dots))}_{n \uparrow f}$ ，也就是将 f 应用 n 次。

用 λ -演算编程 – 邱奇编码 – 自然数 – 构造

自然数的编码也可以从构造和使用的角度来考虑。

$$\bar{0} \stackrel{\text{def}}{=} \lambda f. \lambda x. x$$

$$S \stackrel{\text{def}}{=} \lambda n. \lambda f. \lambda x. f(nfx)$$

$\bar{0}$ 和后继函数 S 能构造出所有自然数的编码。

$$S^n \bar{0} \rightarrow_{\beta}^* \bar{n}$$

用 λ -演算编程 – 邱奇编码 – 自然数 – 使用

(可选) 怎么理解自然数的邱奇编码的“使用”呢？

$\bar{n} = \lambda f. \lambda x. f^n x$ 传入 f 和 x 后，将自然数 n “折叠”成一个值 M 。

1. 如果 $n = 0$, $M = x$;
 2. 如果 n 是 m 的后继³, 且 $\bar{m}fx = N$, $M = fN$ 。
- 熟悉自然数的[数学归纳法](#)的同学会发现，这个跟它很像；并且还很熟悉 *Rocq*⁴的同学会发现，这个就是 *Nat* 的 *induction principles*。
 - 熟悉 *recursion scheme* 的同学会发现，这个就是 *catamorphism*。

³即 $n = m + 1$ 。

⁴*Rocq* 定理证明器，从前叫作 *Coq*。

用 λ -演算编程 – 邱奇编码 – 自然数 – 运算

如果能理解自然数邱奇编码的原理，不难定义自然数的运算。

以下仅作简单展示，大家可自行思考（你可以给出其他定义吗?）。

$$\text{add} \stackrel{\text{def}}{=} \lambda n. \lambda m. \lambda f. \lambda x. \underline{n f (m f x)}$$

$$\text{mult} \stackrel{\text{def}}{=} \lambda n. \lambda m. \lambda f. \underline{n (m f)}$$

$$\text{iszero} \stackrel{\text{def}}{=} \lambda n. \underline{n (K F) T}$$

经典习题：定义 pred 函数，使得 $\text{pred } \bar{n} = \overline{n \dot{-} 1}$ ，其中 $\dot{-}$ 运算符定义如下：

$$n \dot{-} m := \begin{cases} 0 & \text{if } n - m \leq 0 \\ n - m & \text{else} \end{cases}$$

用 λ -演算编程

邱奇编码的介绍就到这里。

- 如果大家对更多数据结构的邱奇编码感兴趣，可以看看[维基百科 - Church encoding](#)。
- 邱奇编码并不完美，也有其他编码方式。
 - Mogensen–Scott encoding
 - Parigot encoding

语法糖

书写 lambda calculus 其实不太方便。

- 函数总是只有一个参数；
- 没法直接定义一个变量；
- 参数即使没被使用，也必须是个变量；
- 等等

很多时候会用一些语法糖来简化书写，最典型的有以下几个：

$$(\text{let } x = M \text{ in } N) \stackrel{\text{def}}{=} (\lambda x.N)M$$

$$\lambda xy \cdots z.M \stackrel{\text{def}}{=} \lambda x.\lambda y.\cdots\lambda z.M$$

$$\lambda_.M \stackrel{\text{def}}{=} \lambda x.M \quad x \text{ 不是 } M \text{ 中的自由变量}$$

λ-演算的扩展

我们不会想要用 λ-演算编码所有数据类型，这在真实的硬件上非常低效。我们可以在内存中直接存储整数和布尔值，也可以用指针或者成块的内存表示对和列表。

这些数据类型可以直接作为一种扩展加入语言中，例如可以直接加入布尔值和自然数。

$$\begin{array}{ll} M, N ::= & \dots \\ & | \text{true} \mid \text{false} \quad \textit{boolean} \\ & | \text{if } M \text{ then } N_1 \text{ else } N_2 \\ & | n \quad \textit{natural number} \\ & | M \text{ op } N \quad \textit{natural number operation} \end{array}$$

其中 $\text{op} \in \{+, -, *, /, =, \neq, \dots\}$

λ-演算的扩展 – 例子

新增语法可以和 λ-演算和谐共存。

$$\text{church_to_boolean} \stackrel{\text{def}}{=} \lambda b. b \text{ true false}$$

$$\text{boolean_to_church} \stackrel{\text{def}}{=} \lambda b. \text{if } b \text{ then } T \text{ else } F$$

$$\text{church_to_nat} \stackrel{\text{def}}{=} \lambda n. n(\lambda m. m + 1)0$$

$$\text{square_nat} \stackrel{\text{def}}{=} \lambda n. n * n$$

经典练习：在整个分享结束后，编写一个 `nat_to_church`⁵。

⁵现在你可能还没有完成这个练习的知识。

λ -演算的形式语义

我们说 λ -演算不仅是一个理想化的程序设计语言，还是一个“形式系统”。

我将非常简单（但会保证精确）地过一遍 λ -演算的形式语义。

λ -演算的形式语义 – α -等价

我们先从所谓的 α -等价开始，之前我们提到过：

$$\lambda x.xx = \lambda y.yy$$

λ-演算的形式语义 – α-等价

我们先从所谓的 α-等价开始，之前我们提到过：

$$\lambda x.xx = \lambda y.yy$$

这很显然，但考虑更加复杂一些的情形呢？

$$\lambda x.\lambda x.x \neq \lambda y.\lambda x.y$$

$$\lambda x.\lambda x.x = \lambda x.\lambda y.y$$

λ -演算的形式语义 – α -等价

$$\frac{y \notin \text{FV}(M)}{\lambda x.M = \lambda y.M[y/x]} \quad (\alpha)$$

$$\frac{M = M'}{\lambda x.M = \lambda x.M'} \quad (\xi)$$

$$\frac{M = M' \quad N = N'}{MN = M'N'} \quad (\text{cong})$$

$$\frac{}{M = M} \quad (\text{refl})$$

$$\frac{N = M}{M = N} \quad (\text{sym})$$

$$\frac{M_1 = M_2 \quad M_2 = M_3}{M_1 = M_3} \quad (\text{trans})$$

最核心的规则就是 (α) ，它表示将 $\lambda x.M$ 中的 x 替换为 y 之后获得的 λ -项和原来的项等价。但是替换如何定义很重要，我们会在后面简单讨论替换的精确定义。

λ -演算的形式语义 – 操作语义

λ -演算的形式语义有很多种，但现在通常我们会定义小步操作语义（small-step operational semantics），因为它展现了 λ -演算的计算过程。

$$\frac{}{(\lambda x.M)N \rightarrow_{\beta} M[N/x]} \quad (\beta)$$

$$\frac{M \rightarrow_{\beta} M'}{\lambda x.M \rightarrow_{\beta} \lambda x.M'} \quad (\xi)$$

$$\frac{M \rightarrow_{\beta} M'}{MN \rightarrow_{\beta} M'N} \quad (\text{cong1})$$

$$\frac{N \rightarrow_{\beta} N'}{MN \rightarrow_{\beta} MN'} \quad (\text{cong2})$$

- 应用 (β) 规则又被称为做 β -规约（reduction）， $(\lambda x.M)N \rightarrow_{\beta}$ 又被称为 β -redex。不存在 redex 的表达式被称为规范形式（normal form）。
- （Church-Rosser 定理）如果 $M \rightarrow_{\beta}^* M_1$ 和 $M \rightarrow_{\beta}^* M_2$ ，那么存在项 N ，有 $M_1 \rightarrow_{\beta}^* N$ 和 $M_2 \rightarrow_{\beta}^* N$ 。

λ-演算的形式语义 – 替换

简单看一下如何定义 $M[N/x]$ 。

$$x[N/x] \stackrel{\text{def}}{=} N$$

$$y[N/x] \stackrel{\text{def}}{=} y \quad \text{if } x \neq y$$

$$(M_1 M_2)[N/x] \stackrel{\text{def}}{=} (M_1[N/x])(M_2[N/x])$$

$$(\lambda x.M')[N/x] \stackrel{\text{def}}{=} \underline{\lambda x.M'}$$

$$(\lambda y.M')[N/x] \stackrel{\text{def}}{=} \lambda y.M'[N/x] \quad \text{if } x \neq y, \underline{y \notin \text{FV}(N)}$$

$$(\lambda y.M')[N/x] \stackrel{\text{def}}{=} \underline{\lambda y'.M'[y'/y]}[N/x] \quad \text{if } x \neq y, \underline{y \in \text{FV}(N)}, \underline{y' \text{ fresh}}$$

λ-演算的形式语义 – 替换

最后一条规则

$$(\lambda y.M')[N/x] \stackrel{\text{def}}{=} \underline{\lambda y'.M'[y'/y]}[N/x] \quad \text{if } x \neq y, \underline{y \in \text{FV}(N)}, \underline{y' \text{ fresh}}$$

可以处理以下情形。

$$\begin{aligned} \underline{(\lambda y.x)y} &\rightarrow_{\beta} (\lambda y.x)[y/x] = \lambda y'.y \\ &\neq \lambda y.y \end{aligned}$$

λ -演算的形式语义 – 自由变量

最后，自由变量的定义比较简单。

$$\text{FV}(x) = \{x\}$$

$$\text{FV}(\lambda x.M) = \text{FV}(M) \setminus \{x\}$$

$$\text{FV}(MN) = \text{FV}(M) \cup \text{FV}(N)$$

λ-演算的求值策略

你可能经常会听到以下词汇：

- 某个语言的求值是“eager”或“lazy”的
- 某个语言是“strict”/“non-strict”的

这些词都跟求值策略有关。

λ-演算的求值策略 – Call-by-Value

Call-by-value (eager 求值)，函数调用必须传入一个“值”。

- 我们首先定义什么叫“值”，在 λ-演算中，值就是函数。

$$v ::= \lambda x.M$$

- 然后限定求值顺序。

$$\frac{M \rightarrow_{\beta} M'}{MN \rightarrow_{\beta} M'N} \text{ (cong1)}$$

$$\frac{N \rightarrow_{\beta} N'}{\underline{v}N \rightarrow_{\beta} \underline{v}N'} \text{ (cong2)}$$

$$\frac{}{(\lambda x.M)\underline{v} \rightarrow_{\beta} M[\underline{v}/x]} (\beta)$$

λ-演算的求值策略 – Call-by-Name

Call-by-name, 函数参数总是原样传入函数, 直到真正被用到时才求值。

$$\frac{M \rightarrow_{\beta} M'}{MN \rightarrow_{\beta} M'N} \text{ (cong1)}$$

$$\frac{}{(\lambda x.M)N \rightarrow_{\beta} M[N/x]} \text{ } (\beta)$$

没有 (cong2) 和 (ξ) 规则。

在这种求值顺序下, 函数的参数只有真正被使用的时候才会被求值。

λ-演算的求值策略 – Call-by-Name

实际上，没有什么现代语言在使用 Call-by-name，因为如果我们多次使用同一个函数参数，那么这个参数会被求值多次。

- Call-by-name 下，以下 λ-项需要
 四步求值。
- 而在 Call-by-value 下，同样的项
 只需三步求值。

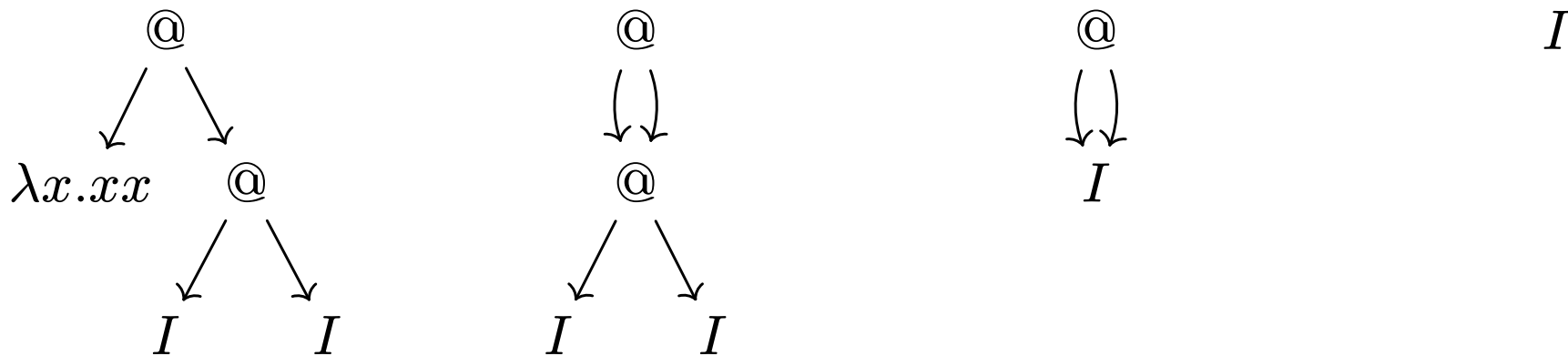
$$\begin{aligned} \underline{(\lambda x.xx)(II)} &\rightarrow_{\beta} \underline{(II)}(II) \\ &\rightarrow_{\beta} \underline{I(II)} \\ &\rightarrow_{\beta} \underline{(II)} \\ &= I \end{aligned}$$

$$\begin{aligned} (\lambda x.xx)\underline{(II)} &\rightarrow_{\beta} \underline{(\lambda x.xx)I} \\ &\rightarrow_{\beta} \underline{II} \\ &= I \end{aligned}$$

λ-演算的求值策略 - Call-by-Need

如果我们对 Call-by-name 做一个优化，让同一个参数的多次使用的求值可以共享，就变成了 Call-by-need。

可以用“图规约”来理解 call-by-need，还是以 $(\lambda x.xx)(II)$ 为例子。



λ-演算的求值策略

我们介绍了常见的三种求值策略：

- Call-by-value
- Call-by-name
- Call-by-need

那么一些常见程序设计语言中的说法对应这里的哪种求值策略呢？

Eager	Call-by-value
Lazy	Call-by-name/need

Strict	Call-by-value
Non-strict	Call-by-name/need

λ-演算的求值策略 – Strict vs. Non-strict

Lazy 和 eager 比较好理解：

- Lazy 的求值策略，项只有在真正被用到的时候才被求值；
- Eager 的求值策略，总是先将参数求值完毕再传入。

可什么是 strict 和 non-strict 呢？

- 在 strict 的语言中，一定有 $f\perp = \perp$ ；
- 在 non-strict 的语言中，可以存在函数 f ，使得 $f\perp \neq \perp$ ，比如 $f = KI$ 。

这里的 $\perp$ 就是不停机或者出错的意思。

λ-演算的求值策略 – 在 Nix 里试试

Nix 是 call-by-need, lazy, 和 non-strict 的语言。

- Call-by-need

```
1 (x: x x) (builtins.trace "once" (x: x))
```

nix

- Lazy

```
1 (x: "value1") (builtins.trace "not evaluated" "value2")
```

nix

- Non-strict:

```
1 (x: "ok") (throw "error")
```

nix

不动点 - 引入

现在我们来介绍一个经典且极其重要的 λ -演算构造，不动点。

为了方便理解，我用前述的扩展了布尔值和自然数的 λ -演算来做引入。

让我们在扩展的 λ -演算上尝试定义一个阶乘函数：

$$\text{factorial} \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } \text{factorial } (n - 1) * n$$

你看出什么问题了吗？

不动点 - 引入

现在我们来介绍一个经典且极其重要的 λ -演算构造，不动点。

为了方便理解，我用前述的扩展了布尔值和自然数的 λ -演算来做引入。

让我们在扩展的 λ -演算上尝试定义一个阶乘函数：

$$\text{factorial} \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } \text{factorial } (n - 1) * n$$

你看出什么问题了吗？

循环定义！

不动点 - 引入

$\text{factorial} \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } \text{factorial } (n - 1) * n$

让我们把刚才的错误定义拆解改写一下：

$\text{factorial} \stackrel{\text{def}}{=} \text{factorial_gen } \text{factorial}$

$\text{factorial_gen} \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } f(n - 1) * n$

不动点 - 引入

$\text{factorial} \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } \text{factorial } (n - 1) * n$

让我们把刚才的错误定义拆解改写一下：

$\text{factorial} \stackrel{\text{def}}{=} \text{factorial_gen } \text{factorial}$

$\text{factorial_gen} \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } f(n - 1) * n$

要定义 factorial，我们就要找到某个函数 f ，使得 f 的语义和 factorial_gen f 的语义相同，然后将 f 作为 factorial 的定义。

不动点 - 引入

$\text{factorial} \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } \text{factorial } (n - 1) * n$

让我们把刚才的错误定义拆解改写一下：

$\text{factorial} \stackrel{\text{def}}{=} \text{factorial_gen } \text{factorial}$

$\text{factorial_gen} \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } f(n - 1) * n$

要定义 factorial，我们就要找到某个函数 f ，使得 f 的语义和 factorial_gen f 的语义相同，然后将 f 作为 factorial 的定义。

我们要找的 f 叫作 factorial_gen 的不动点 (fixed-point)。

不动点

数学上，函数 $f : A \rightarrow B$ 的不动点指的是某个 $c \in A \cap B$ ，满足：

$$f(c) = c$$

同样也可以对 λ -演算中的函数定义不动点，对于 λ -项 f ，它的不动点也是一个项 c 满足： $f(c)$ 和 c 的语义相同，还是写作 $f(c) = c$ 。

不动点

数学上，函数 $f : A \rightarrow B$ 的不动点指的是某个 $c \in A \cap B$ ，满足：

$$f(c) = c$$

同样也可以对 λ -演算中的函数定义不动点，对于 λ -项 f ，它的不动点也是一个项 c 满足： $f(c)$ 和 c 的语义相同，还是写作 $f(c) = c$ 。

但是，什么是 λ -项之间的等价关系 ($=$)？我们希望这个等价关系刻画了 λ -项的语义等价。

不动点 =- λ -项的等价关系

(简单看一眼吧)

$$\frac{y \notin \text{FV}(M)}{\lambda x.M = \lambda y.M[y/x]} \quad (\alpha)$$

$$\frac{}{(\lambda x.M)N = M[N/x]} \quad (\beta)$$

$$\frac{x \notin \text{FV}(M)}{\lambda x.Mx = M} \quad (\eta)$$

$$\frac{M = M'}{\lambda x.M = \lambda x.M'} \quad (\xi)$$

$$\frac{M = M' \quad N = N'}{MN = M'N'} \quad (\text{cong})$$

$$\frac{}{M = M} \quad (\text{refl})$$

$$\frac{N = M}{M = N} \quad (\text{sym})$$

$$\frac{M_1 = M_2 \quad M_2 = M_3}{M_1 = M_3} \quad (\text{trans})$$

(η) 规则的存在使该等价关系有函数外延性 (*extensionality*)，即 $(\forall N.M_1N = M_2N)$ 蕴含 $M_1 = M_2$ 。

不动点 - λ -项的等价关系

换一种方式解释，这个等价关系描述了，只要两个 λ -项，能求值成同一个项（包含 α -等价和外延性），那么它们就等价。因此这种等价是语义上的。

对外延性，组合 (ξ) ， (cong1) ， (cong2) 和下面的 (η) 规则，定义 $\rightarrow_\eta$ ：

$$\frac{x \notin \text{FV}(M)}{\lambda x.Mx \rightarrow_\eta M} (\eta)$$

组合 (ξ) ， (cong1) ， (cong2) ， (β) ， (η) 定义 $\rightarrow_{\beta\eta}$ 。然后我们就可以说，

$$M_1 = M_2 \text{ 当且仅当存在 } N, M_1 \rightarrow_{\beta\eta}^* N \text{ 和 } M_2 \rightarrow_{\beta\eta}^* N。$$

我们定义的 $=$ 是包含 (β) 和 (η) 的 $=_{\beta\eta}$ ，还可以定义仅包含 (β) 的 $=_\beta$ 和仅包含 (η) 的 $=_\eta$ 。

不动点 - 不动点组合子

如何获得 factorial_gen 的不动点？

不动点 - 不动点组合子

如何获得 `factorial_gen` 的不动点？

如何获得任意函数 f 的不动点？

不动点 - 不动点组合子

在 λ -演算中，存在不动点组合子 Y ，使得对于任意函数 f ，都有：

$$f(Y f) = Y f$$

还有其他不动点组合子，比如 Z ，用在 *strict* 的语言中。

不动点 - 不动点组合子

在 λ -演算中，存在不动点组合子 Y ，使得对于任意函数 f ，都有：

$$f(Y f) = Y f$$

从计算/操作语义的角度来看：

$$\begin{aligned} Y f &\rightarrow_{\beta} f(Y f) \\ &\rightarrow_{\beta} f(f(Y f)) \\ &\rightarrow_{\beta}^* f(\cdots(f(Y f))\cdots) \end{aligned}$$

还有其他不动点组合子，比如 Z ，用在 *strict* 的语言中。

不动点 - 不动点组合子

$$Yf \rightarrow_{\beta}^* f(Yf)$$

现在我们可以先假定这个魔法般的组合子存在，但是它为什么不会无限递归？

error: infinite recursion encountered

不动点 - 不动点组合子

$$Y f \rightarrow_{\beta}^* f(Y f)$$

现在我们可以先假定这个魔法般的组合子存在，但是它为什么不会无限递归？

error: infinite recursion encountered

答案是 **call-by-need**，在 lazy 的语言中 $Y f$ **可以不**无限递归！

在 call-by-need 的求值策略下，只要我们不在 f 中总是使用 $Y f$ ，而是有递归终止条件，就不会无限递归。

不动点 - 例子

$$\text{factorial} \stackrel{\text{def}}{=} Y \text{ factorial_gen}$$

$$\text{factorial_gen} \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } f(n - 1) * n$$

$$\text{factorial} = \text{factorial_gen } Y$$

$$\rightarrow_{\beta}^* \text{factorial_gen } (\text{factorial_gen } Y)$$

$$= \text{factorial_gen } \text{factorial}$$

$$= (\lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } f(n - 1) * n) \text{ factorial}$$

$$\rightarrow_{\beta}^* \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else factorial } (n - 1) * n$$

$$\text{factorial} = \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else factorial } (n - 1) * n$$

理解 Nix 语法和库

理解不动点和 **call-by-name** 的概念有助于我们理解一些 Nix 语法和库，因为它们大量使用不动点。

- `lib.fix` 函数
- Nix `let` 语法
- Nix `rec` 语法
- Nixpkgs overlays 机制
- NixOS module

Call-by-name 在这些机制的理解中相当重要。

理解 Nix 语法和库 – lib.fix

Nix 的 lib.fix 就是一个不动点组合子，和 Y 等价。

```
1 let
2   factorial_gen = f: n: if n == 0 then 1 else f (n - 1) * n;
3   factorial = lib.fix factorial_gen;
4 in
5   factorial 5 # 120
```

nix

因为 Nix 的 let 直接支持递归，lib.fix 的定义如下。

```
1 fix = f: let x = f x; in x
```

nix

理解 Nix 语法和库 – let

```
1 let factorial = n: if n == 0 then 1 else factorial (n - 1) * n;
2 in factorial 5 # 120
```

nix

可以理解为：

```
1 let factorial = lib.fix (f: n: if n == 0 then 1 else f (n - 1) * n);
2 in factorial 5 # 120
```

nix

理解 Nix 语法和库 – let

Nix 的 let 还支持互递归 (mutual recursion):

```
1 let odd = n: if n == 0 then false else even (n - 1);  
2     even = n: if n == 0 then true  else odd  (n - 1);  
3 in odd 100
```

nix

可以理解为:

```
1 let fs = { odd = n: if n == 0 then false else fs.even (n - 1);  
2           even = n: if n == 0 then true  else fs.odd  (n - 1); };  
3 in fs.odd 100
```

nix

理解 Nix 语法和库 – rec

```
1  rec { a = 1; b = a + 1; } # { a = 1; b = 2; }
```

nix

可以理解为：

```
1  lib.fix (self: { a = 1; b = self.a + 1; })
```

nix

理解 Nix 语法和库 – Nixpkgs overlays 机制

```
1 final: prev: { ... }
```

nix

整个 nixpkgs 其实就是由一堆 overlays 生成的⁶。以下是经过修改便于理解的代码。

```
1 let extends = f: overlay
```

nix

```
2     final: let prev = f final; prev // overlay final prev;
```

```
3     toFix = lib.foldl' extends (self: { }) allOverlays;
```

```
4 in lib.fix toFix
```

final 是最终的 nixpkgs，而 prev 是上一个 overlay 生成的 nixpkgs，overlay 被传入这两个参数后与 prev 组合。最终 final 是通过 fix 被传入的。

⁶[nixos/nixpkgs - pkgs/top-level/stage.nix – L312-L336](#)

理解 Nix 语法和库 – NixOS module - config

NixOS module 中也有一些特别的和不动点有关的机制，需要特别关注它们的用法，这里做一个简介。

理解 Nix 语法和库 – NixOS module

Module 的参数中有 config，表示 evalModules 将所有 module 合并后获得的结果。所以这个结果是一个不动点。

```
1 (lib.evalModules { modules = [nix
2   ({ config, ... }: {
3     options = rec { a = lib.mkOption { type = lib.types.int; }; b = a; };
4     a = 1; b = config.a; })
5   ({ ... }: { a = lib.mkForce 2; })
6 ]; }).config # { a = 2; b = 2; }
```

理解 Nix 语法和库 – NixOS module - `_module.args`

Module 中可以包含 `_module.args`，而 `_module.args` 又会变成所有 module 的参数。

```
1 { pkgs, ... }: { _module.args.pkgs = ...; }
```

nix

在这个 module 中 `pkgs` 既是它的输入又是它的输出，同样是一个不动点。

理解 Nix 语法和库 – NixOS module - imports

Module 可以 import 其他 module:

```
1 { ... }: { imports = [ ./path/to/other/modules.nix ]; }
```

nix

如果 imports 路径包含 evalModules 的结果, 就会 infinite recursion。

```
1 { config, ... }:  
2 { imports = [ config.module.defined.in.options ]; } # bad
```

nix

```
1 { pkgs, ... }:  
2 { imports = [ "${pkgs}/nixos/modules/path/to/module.nix" ]; } # bad
```

nix

如果要向 modules 中传入路径以在 imports 中使用, 需要 specialArgs 而不是 _module.args。

扩展知识 - 不动点组合子

首次接触 λ -演算的同学可能很疑惑，我们前面一直假设不动点组合子 Y 存在，但 Nix 是用内置的 `let` 语法实现 `lib.fix` 的， Y 真的存在吗？

$$Y f \rightarrow_{\beta}^* f(Y f)$$

扩展知识 - 不动点组合子

首次接触 λ -演算的同学可能很疑惑，我们前面一直假设不动点组合子 Y 存在，但 Nix 是用内置的 `let` 语法实现 `lib.fix` 的， Y 真的存在吗？

$$Y f \rightarrow_{\beta}^* f(Y f)$$

答案当然是存在，让我们一步步重新发现 Y 。

扩展知识 - 不动点组合子

$$Y f \rightarrow_{\beta}^* f(Y f)$$

为了定义 $Y f$ ，我们需要将 $Y f$ 自身传入 f 。在 λ -演算中，我们没有递归，如何获得函数本身？

思考：有没有一种办法在一个函数中获得函数自身？

扩展知识 - 不动点组合子

$$Yf \rightarrow_{\beta}^* f(Yf)$$

为了定义 Yf ，我们需要将 Yf 自身传入 f 。在 λ -演算中，我们没有递归，如何获得函数本身？

思考：有没有一种办法在一个函数中获得函数自身？

当然有！

$$(\lambda x.xx)(\lambda s.M)$$

在 $(\lambda s.M)$ 这个函数中， s 就是 $(\lambda s.M)$ 自身。

扩展知识 - 不动点组合子

$$Yf \rightarrow_{\beta}^* f(Yf)$$

s 就是 $(\lambda s.M)$ 自身，这样的话，在 M 中，我们就有 $ss = M$ 。

$$(\lambda x.xx)(\lambda s.M) = M$$

看出来了吗？

扩展知识 - 不动点组合子

$$Y f \rightarrow_{\beta}^* f(Y f)$$

s 就是 $(\lambda s.M)$ 自身，这样的话，在 M 中，我们就有 $ss = M$ 。

$$(\lambda x.xx)(\lambda s.M) = M$$

看出来了吗？

令 $M = fM = f(ss)$ ，我们有：

$$M = (\lambda x.xx)(\lambda s.f(ss)) = fM$$

M 就是 f 的不动点。

扩展知识 - 不动点组合子

将 f 抽象出来，我们就得到了 Y 。

$$M = (\lambda x.xx)(\lambda s.f(ss))$$

$$Y \stackrel{\text{def}}{=} \lambda f.(\lambda x.xx)(\lambda s.f(ss))$$

$$\stackrel{\text{def}}{=} \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

第二种写法是更常见的。

非常容易验证：

$$Yf \rightarrow_{\beta}^* f(Yf)$$

希尔伯特可判定性问题与邱奇-图灵论题



图 2: 大卫·希尔伯特

1928 年，数学家大卫·希尔伯特（David Hilbert）提出了“希尔伯特计划”，试图构建一套公理集，为全部的数学提供一个安全的理论基础。

这个基础应该能够实现：

- 所有数学的形式化；
- 完备性：所有真命题都能被证明；
- 一致性：不可能导出矛盾；
- 可判定性：能用**算法**判定命题是真命题还是假命题。

希尔伯特可判定性问题与邱奇-图灵论题



图 3: 库尔特·哥德尔

但希尔伯特的宏伟设想被证明是无法实现的。

1931 年，库尔特·哥德尔（Kurt Gödel）提出哥德尔不完备定理。证明了计划中的完备性是不可能做到的。

希尔伯特可判定性问题与邱奇-图灵论题

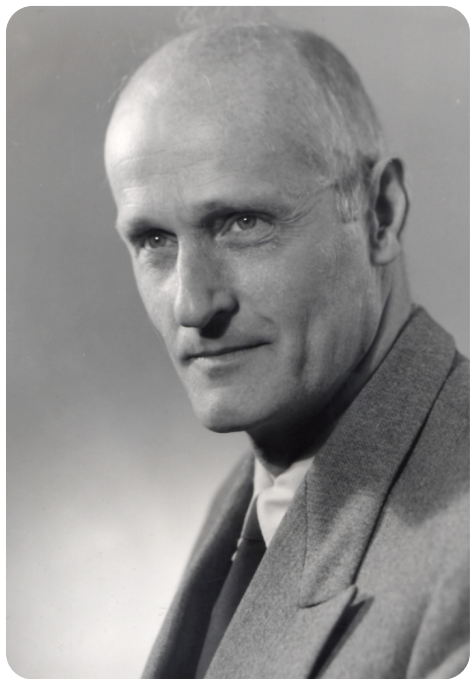


图 4: 斯蒂芬·克莱尼

哥德尔为了证明不完备定理，定义了原始递归函数类（Primitive Recursive Functions），他的学生斯蒂芬·克莱尼（Stephen Kleene）等人将其扩充，提出了一般递归函数类（General Recursive Functions）。如今，一般递归函数又被称为**直觉可计算**函数。

希尔伯特可判定性问题与邱奇-图灵论题



图 5: 艾伦·图灵

随后，有两个重要的工作独立地提出了**计算**的模型，证明了“可判定性”也是不可能做到的，顺便奠定了**计算机**的基础。

- 艾伦·图灵 (Alan Turing) 提出了图灵机否定了“可判定性”。
- 阿隆佐·邱奇用 λ -演算作为计算模型否定了“可判定性”。

希尔伯特可判定性问题与邱奇-图灵论题

20 世纪 30 年代，人们从算术、函数抽象和机械计算三个完全不同的角度提出了三种计算模型：

一般递归函数

λ -演算

图灵机

并证明它们刻画了同一类函数，它们具有同等的计算能力。

这种惊人的一致性促成了邱奇-图灵论题：

“可计算”这一非形式化概念可以由这些等价的形式模型准确刻画。

λ-演算的实现

或者说，如何在硬件上模拟 λ-演算？

- 在操作语义中，λ-演算的操作语义是用“替换”来实现的，但替换是语法的操作。
- 另一种方式使用是类似 Python 或者 Scheme 的基于环境的语义⁷。

$$\text{Closure} = \text{Code} + \text{Environment}$$

$$\text{Environment} : \text{Variables} \rightarrow \text{Values}$$

如果你学习过 λ-演算的指称语义 (*denotational semantics*)，其实基于环境的语义就是 λ-演算指称语义的一种实现。

⁷可见 [R7RS specification](#)。

λ-演算的实现 - 闭包

Closure = Code + Environment

Environment : Variables $\rightarrow$ Values

等等？什么是闭包（closure）？为什么我们讲完了 λ-演算却不需要讲闭包？

λ-演算的实现 - 闭包

$\text{Closure} = \text{Code} + \text{Environment}$

$\text{Environment} : \text{Variables} \rightarrow \text{Values}$

等等？什么是闭包（closure）？为什么我们讲完了 λ-演算却不需要讲闭包？

闭包其实就是词法作用域（lexical scope）的一种实现方式。

λ-演算的实现 - 闭包

$\text{Closure} = \text{Code} + \text{Environment}$

$\text{Environment} : \text{Variables} \rightarrow \text{Values}$

等等？什么是**闭包 (closure)**？为什么我们讲完了 λ-演算却不需要讲闭包？

闭包其实就是词法作用域 (lexical scope) 的一种实现方式。

等等？什么是**词法作用域**？为什么我们讲完了 λ-演算却不需要讲作用域？

λ-演算的实现 - 闭包

Closure = Code + Environment

Environment : Variables $\rightarrow$ Values

等等？什么是**闭包 (closure)**？为什么我们讲完了 λ-演算却不需要讲闭包？

闭包其实就是词法作用域 (lexical scope) 的一种实现方式。

等等？什么是**词法作用域**？为什么我们讲完了 λ-演算却不需要讲作用域？

因为词法作用域已经体现在替换中。

$$\frac{}{(\lambda x.M)N \rightarrow_{\beta} M[N/x]} (\beta)$$

λ-演算的实现 - Call-by-Need

Nix 是 call-by-need 的语言，如何实现 call-by-need 呢？

- 一种方式是继续使用基于环境的语义，但我们向值中加入所谓的“thunk” – 延迟计算。

$$(\lambda x.M)N$$

N 被打包成一个 thunk t ，对 M 求值时，环境为 $\{x \mapsto t\}$ ，当 t 被使用时才求值 N 。

- 另一种方式是图规约，编译到 G-machine⁸及其衍生技术。

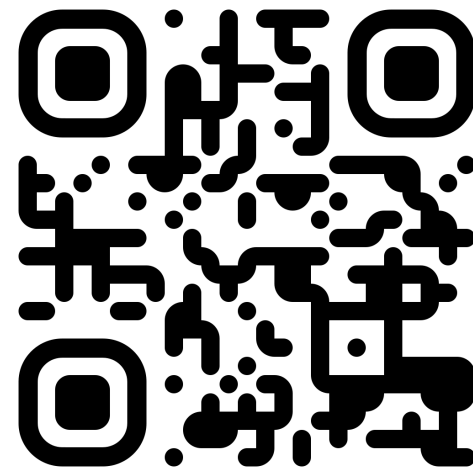
Haskell 使用的就是 STG machine (Spineless Tagless G-machine)。

⁸感兴趣的同学可以学习 Simon Peyton Jones 的《Implementing functional languages: a tutorial》

谢谢!



获取本幻灯片



在线 λ -演算解释器

lambdacalc.dev